



Trabajo Final de Ingeniería

Sistema de Mantenimiento predictivo de equipos informáticos

Log de prompts — Entrega de sistema

Alumno: Pettini Patricio Ezequiel

Localización: Norte Boulogne

Profesores: Romero Martín Miguel, Weingand Gastón Matías

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martin - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

Índice

1 • Qué registra este documento	4
2 • Las decisiones que tomé	5
2.1 • Arquitectura	5
2.2 • El motor predictivo	5
2.3 • Verificación y calidad	6
2.4 • Qué supone cada decisión en el código	7
3 • Correcciones que hice sobre lo que la herramienta produjo	9
4 • Lo que hice a mano	11
El modelo de Enterprise Architect	11
El sábado 5 de septiembre	11
El documento dejó de generarse	11
El diseño de la interfaz	12
Lo que audite leyendo, no ejecutando	12
5 • Y del otro lado: qué encontró la herramienta	13
6 • Apéndice: las decisiones, una por una	14
6.1 • La arquitectura base	14
C# y ASP.NET Core, en lugar del Java y Spring Boot que fija el documento • ADR 0002 ...	14
Dos bases de datos con cadenas separadas • ADR 0003	15
ADO.NET con DAO explícito, no Entity Framework • ADR 0004	15
Multi-organización desde el modelo de datos • ADR 0005	16
Identidad propia, sin gestor externo • ADR 0014	16
6.2 • El dominio y el motor predictivo	17
Proveedor de IA conmutable, con asignación de respaldo • ADR 0006	17
Reglas predictivas parametrizadas, resueltas con estrategias • ADR 0007	18

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martin - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

El motor se dispara solo · ADR 0011.....	18
La máquina de estados de la incidencia vive en la BLL · ADR 0012.....	19
Reportes en PDF generados en el servidor · ADR 0013	19
6.3 · Seguridad y operación	19
Cifrado reversible, separado del hash de contraseñas · ADR 0010	19
Patrones de resiliencia, acotados a dos puntos · ADR 0009.....	20
El nivel de verificación es superior al estándar, a propósito · ADR 0008	20
Métricas en formato Prometheus, sin stack propio · ADR 0015	20
6.4 · Decisiones de código que no llevan ADR.....	21
6.5 · Cómo audito la construcción	21

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

1 · Qué registra este documento

Registra las consultas asistidas por inteligencia artificial que produjeron el sistema y su documentación técnica: qué se pidió, qué se aceptó, qué se corrigió y cómo se verificó cada cosa antes de darla por hecha.

La mitad de negocio —el plan y el presupuesto financiero— tiene su propio registro, en la carpeta de esa entrega. Las intervenciones que afectaron a las dos mitades aparecen en los dos, porque afectaron a las dos.

Herramienta: Claude Code (Claude Opus 5), usada para analizar el documento del Trabajo Final, diseñar el esquema de base, construir el sistema en sus cinco capas, y redactar la documentación técnica.

Este log es el resumen ejecutable de dos documentos que lo preceden y que traen el detalle: el *Anexo de trazabilidad del uso de IA*, que registra el uso por fase, y PROMPTS-CRITICOS.md, que documenta con el modelo C4 las consultas que cambiaron decisiones de arquitectura y derivan en los ADR.

Quién decidió qué, y sabiendo qué. La herramienta construyó y analizó; la dirección, los criterios de calidad, las correcciones y los cortes de alcance son míos. La sección 2 lista las decisiones, con su fundamento; la sección 3, lo que corregí sobre lo producido; la sección 4, lo que construí a mano; y el apéndice de la sección 6 ubica todo en la cronología.

Ese apéndice agrega lo que un pedido no muestra por sí solo: qué había que saber para hacerlo. Decidir dos bases físicas separadas supone saber qué se pierde con eso; rechazar un gestor de identidad federada supone saber qué resuelve y qué cuesta. Va en la columna «con qué base», y donde no hubo conocimiento previo, lo dice.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	Fecha: 13/06/2026
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

2 · Las decisiones que tomé

2.1 · Arquitectura

Seis decisiones, todas con su registro de decisión de arquitectura, y varias apartándose de lo que fija el documento o eligiendo la opción más costosa:

Decisión	Por qué	ADR
C# / ASP.NET Core en lugar del Java / Spring Boot que fija el documento	Es el lenguaje de la carrera y el del proyecto de referencia de 3.º año: permite reutilizar el modelo de capas que evalúa la cátedra, y es el código que voy a tener que sostener en la defensa	0002
Dos bases físicas separadas, negocio y servicio	Asumiendo la pérdida de integridad referencial entre las dos. La separación es el punto: un DAO de negocio no puede alcanzar la base de seguridad, ni al revés	0003
ADO.NET con DAO explícito, no Entity Framework	Asumiendo el código repetitivo. El modelo de capas que se evalúa es ése, y un ORM lo esconde	0004
Multi-organización desde el modelo de datos, ampliando el DER del documento	El plan de negocio vende un Plan Partner multi-cliente. Sin la entidad en el modelo, ese plan no se puede sostener	0005
Alcance de la IA: asignación y clasificación, descartando otras opciones propuestas	Son las dos que el sistema puede demostrar sin crédito de API y sin entrenar un modelo propio	0006
Proveedor de IA conmutable entre real y simulado	Ante la falta de crédito de API. Y además es lo que hace demostrable el caso de prueba CP-14, la caída del servicio de IA	0006

Y dos ampliaciones de alcance que rechacé: la gestión de identidad federada —un IAM— y el dígito verificador. La primera agranda el alcance sin que el trabajo la pida; el control de acceso se resuelve con patentes y JWT, que es lo que se evalúa.

2.2 · El motor predictivo

Cuatro decisiones que la herramienta no propuso y que cambian el comportamiento del sistema:

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Decisión	Por qué
La fecha de referencia entra por parámetro; el motor no lee el reloj	Sin esto no existe caso de prueba posible: el resultado cambiaría según el día en que se corran las pruebas. Es la decisión que hace verificable todo el capítulo
Una regla que no se puede evaluar por falta de datos se excluye del promedio ponderado, en lugar de aportar cero	Si aportara cero, el equipo con la ficha incompleta aparecería *menos* riesgoso que el que está bien cargado, que es exactamente al revés de lo que corresponde
La criticidad del equipo multiplica el score, no lo suma	Sumando, un servidor y una impresora con el mismo historial terminan a distancia fija; multiplicando, la criticidad pesa sobre todo el resto
El análisis corre solo, no sólo al registrar una incidencia	Una alerta que depende de que alguien cargue algo no es predictiva

2.3 · Verificación y calidad

Decisión	Por qué
La vara de arquitectura es la del proyecto de 3.º año: bitácora, seguridad, multidioma, criptografía, y el esquema de patentes y familias	Es lo que la cátedra evalúa, y lo conozco de primera mano porque lo construí
La auditoría se hace contra datos, no contra el código	Es la decisión que dio resultado. Los dos hallazgos serios no eran errores sino ausencias, y una ausencia no se ve leyendo código
Auditar el sistema como lo haría un auditor de una empresa grande, y que se entienda como si lo usara alguien de quince años	El sistema se defiende ante alguien que no lo construyó. Si hay que explicarlo, está mal hecho
Los reportes son PDF, no CSV	La propuesta era CSV por ser lo más barato y sin dependencias. Un CSV no lleva encabezado ni los filtros impresos, y no se ve como un entregable
El documento del Trabajo Final se mantiene a mano y el pipeline deja de pisarlo; los diagramas salen del modelo de Enterprise Architect	Trabajé el documento a mano un sábado entero. Un script que lo regenera borra ese trabajo sin avisar

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martin - Weingand, Gastón	Fecha: 13/06/2026
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

2.4 · Qué supone cada decisión en el código

Las tres tablas de arriba dicen qué decidí. Ésta dice qué hubo que resolver en el código por haberlo decidido así, que es lo que separa una preferencia de una decisión de ingeniería: cada una cierra una puerta y abre un problema, y el problema hay que saber dónde se resuelve.

Decisión	Lo que obliga a resolver, y dónde
Dos bases con cadenas separadas (ADR 0003)	El motor deja de poder garantizar las referencias cruzadas. Equipo.id_responsable, Incidencia.id_tecnico y Mantenimiento.id_tecnico son referencias lógicas por GUID, sin clave foránea, y su integridad se valida en la BLL antes de persistir. Cada DAO recibe su cadena y no conoce la otra: ésa es la separación, y es también lo que hace que un JOIN entre negocio y seguridad no exista como opción
ADO.NET con DAO explícito (ADR 0004)	No hay ORM que arme las consultas, así que el modelo de capas queda a la vista: IGenericDao<T> para el CRUD, IMapper<T> para pasar de SqlDataReader a entidad, FactoryDao y FactoryBusiness como fábricas abstractas, y SqlTransactRepository como unidad de trabajo cuando una operación toca varias tablas. Todo parámetro entra como SqlParameter: no hay una sola concatenación de SQL en las cinco capas
Permisos con patentes y familias	Resolver si un usuario tiene un permiso no es leer una fila: es recorrer un árbol. Familia_Familia permite familias dentro de familias, así que el chequeo es un Composite con recorrido en profundidad, y el filtro [RequierePatente("...")] de la API se apoya en él
Aislamiento por organización (ADR 0005)	id_organizacion en toda tabla de negocio, y el filtro aplicado en el DAO a partir del contexto de sesión, no en el controlador. Es deliberado: puesto en el controlador, un endpoint nuevo puede olvidarse de filtrar y nadie se entera; puesto en el DAO, no hay forma de consultar sin filtrar
La máquina de estados de la incidencia vive en la BLL (ADR 0012)	Ni en la base con un CHECK, ni en el frontend deshabilitando botones. Los siete estados de EstadoIncidencia y las transiciones válidas entre ellos se deciden en un solo lugar, que es el que las pruebas pueden recorrer entera
El motor predictivo se dispara solo (ADR 0011)	Deja de alcanzar con evaluar al registrar una incidencia: hay un trabajo programado, y con él aparece el problema de la fecha. Por eso la fecha de referencia entra por parámetro y el motor no lee el reloj: es lo que hace que el resultado de una prueba no dependa del día en que se corre

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Decisión	Lo que obliga a resolver, y dónde
Reglas parametrizadas con estrategias (ADR 0007)	ReglaAlerta.condicion guarda JSON con el tipo de regla y sus parámetros, y cada tipo lo resuelve una estrategia distinta. Agregar una regla es agregar una clase y una fila, no tocar el evaluador
Proveedor de IA conmutable (ADR 0006)	Adapter: IProveedorIA con la implementación real y la simulada detrás de la misma interfaz. El camino de respaldo no es un catch vacío: ante error o tiempo agotado asigna al técnico de menor carga, deja tipo_asignacion = 'RESPALDO', marca la incidencia para revisión y asienta el evento. Es lo que hace demostrable el CP-14
Cifrado reversible separado del hash (ADR 0010)	Son dos problemas distintos y se resuelven con dos primitivas distintas. La contraseña no se recupera nunca: PBKDF2-HMAC-SHA256 con el formato PBKDF2\$iteraciones\$salt\$hash y 600 000 iteraciones, que es lo que recomienda OWASP. La clave de API de la organización sí hay que recuperarla para usarla: AES-256-GCM, con la clave fuera del código y ningún endpoint que la devuelva
Nombres de tablas y columnas	plan y Backup son palabras reservadas de T-SQL, así que la tabla del respaldo se llama Respaldo y la columna del abono plan_comercial. Es el tipo de detalle que no se descubre diseñando: se descubre cuando la consulta falla

Cada fila de esta tabla se puede abrir en la defensa mostrando el archivo. Es a propósito: un registro de prompts que no se puede contrastar contra el código no prueba nada.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Trabajo Final de Ingeniería		Docentes:		Fecha: 13/06/2026
			Romero, Martin - Weingand, Gastón		
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1		
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026	
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0	

3 · Correcciones que hice sobre lo que la herramienta produjo

Con la columna que importa: cómo me di cuenta. Ninguna salió de leer el código.

Fecha	Qué corregí	Cómo me di cuenta	Consecuencia
02/09	El plan que «convenía» no convenía. La pantalla de Organización afirmaba que con 52 equipos convenía pasar al Plan Estándar	Hice la cuenta contra la base: el Inicial daba ARS 101.500 y el Estándar ARS 102.900, con los precios de ese momento. El Estándar recién convenía desde 57 equipos	La pantalla pasa a mostrar la cuenta real, incluso cuando el resultado es que no conviene cambiar al plan más caro. Con los precios revisados de D-14 la conclusión se mantiene: 244.400 contra 246.000, y el Estándar recién conviene desde 55 equipos
02/09	El anexo de trazabilidad no registraba los prompts	Lo revisé y no estaban. Un anexo de trazabilidad sin las consultas no es trazabilidad	Se incorporan las consultas que cambiaron el diseño, y los anexos pasan a archivos separados
03/09	El formato de los documentos no estaba a la altura de lo que se entrega. Tipografía, jerarquía de títulos, tablas y epígrafes no seguían ningún criterio	Los leí completos, uno por uno, como los va a leer el profesor, y los comparé contra el proyecto de 3.º año —que ya pasó por la cátedra— y contra mi propia documentación	Fijé esos dos como patrón y se rehízo el formato entero: capturas reales, normas APA y el paquete completo en Word y PDF
03/09	La columna «Responsable» del Dashboard traía nombres de técnicos en la tabla de equipos con mayor riesgo	Cada fila de esa tabla es un equipo, y un equipo no tiene técnico asignado: lo tienen sus incidencias. El rótulo estaba bien, el dato estaba mal	Se corrigió el dato al responsable del equipo. Queda en D-15
04/09	El motor predictivo no se disparaba solo	Lo reporté mientras la herramienta estaba trabajando en otra cosa, y la evidencia que había a	Se verificó contra la base y era cierto. Sin eso, un requisito quedaba sin cumplir de una forma que ninguna prueba mostraba

UNIVERSIDAD ABIERTA INTERAMERICANA

Facultad de Tecnología Informática



Materia: **Trabajo Final de Ingeniería**

Docentes:

Romero, Martín - Weingand, Gastón

Fecha:
13/06/2026

Alumno: **Pettini, Patricio Ezequiel**

Legajo: **B00072710-T1**

Sede: **Norte**

Comisión: **5 - A**

Turno: **Noche**

Año: **2026**

Sistema de Mantenimiento predictivo de equipos informáticos

Versión: 2.0

Fecha	Qué corregí	Cómo me di cuenta	Consecuencia
		mano parecía indicar lo contrario	
04/09	Los diagramas exportados salían ilegibles: texto cortado y cajas más chicas que su contenido	Los miré uno por uno y mandé las capturas señalando el defecto	Cajas al tamaño del contenido, el DER partido por área, y los 22 diagramas rehechos

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

4 · Lo que hice a mano

No todo paso por la herramienta, y lo que no paso esta aca. Cada linea se puede comprobar en el historial del repositorio.

El modelo de Enterprise Architect

Los veintidos diagramas del documento salen de un modelo que rehice a mano en Enterprise Architect: docs/tecnico/Modelo Enterprise Architect - PredictIT.eap. Un script los habia generado leyendo el esquema real de la base, y esa version sirvio de piso —garantiza que un diagrama entidad-relacion no se desactualice respecto de las tablas—, pero los que se entregan son los que dibuje yo: los tres DER con notacion pata de gallo y ruteo ortogonal, el diagrama de clases partido en tres vistas, y los trece casos de uso.

El script `_diagramas-del-modelo.py` no los dibuja: los exporta del modelo y le pisa al generador los que corresponden. El orden importa y es la razon de que sean dos scripts.

El sabado 5 de septiembre

Cinco commits, ninguna consulta. Trabaje el documento a mano: notacion pata de gallo en los diagramas entidad-relacion, unificacion del formato de los titulos, la escala de los niveles 4 y 5, el diagrama de clases partido en tres y la introduccion de 10.4.7 ajustada a esas tres vistas.

El documento dejo de generarse

Despues de ese sabado decidi que el documento del Trabajo Final se mantiene a mano y que el pipeline no lo pise mas. El script que lo formateaba hacia dos cosas que desharian ese trabajo en una sola corrida: cambiaba cada imagen por la que genera el script de diagramas y volvia a numerar figuras y tablas. Quedo fuera del armado automatico, con el motivo escrito al lado para que nadie lo vuelva a agregar sin leerlo.

Los epigrafes numerados tambien se sacaron a proposito, y esa decision quedo registrada como descartada en el registro de cambios (D-26), no como pendiente.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martin - Weingand, Gastón		Fecha: 13/06/2026
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026	
	Sistema de Mantenimiento predictivo de equipos informáticos				Versión: 2.0

El diseño de la interfaz

Las pantallas se diseñaron en Claude Design antes de que se escribiera una línea del sistema, y ese trabajo es mío. Cuando se propuso rehacer el frontend desde cero, dije que no: ya estaba construido sobre ese diseño y rehacerlo tiraba el trabajo y cambiaba una interfaz que yo ya había validado. El sistema se construyó sobre el frontend existente, por incrementos.

Lo que audite leyendo, no ejecutando

Las seis correcciones de la sección 3 no salieron de una herramienta de análisis: salí a buscarlas. Leí la planilla y no me cerraron los sueldos; hice la cuenta de los dos planes con la calculadora y no daba lo que la pantalla afirmaba; mire los veintidos diagramas exportados uno por uno; y leí los documentos completos como los va a leer el profesor.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martin - Weingand, Gastón		Fecha: 13/06/2026
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026	
	Sistema de Mantenimiento predictivo de equipos informáticos				Versión: 2.0

5 · Y del otro lado: qué encontró la herramienta

La contrapartida, porque el registro no sirve si sólo cuenta una mitad. Los hallazgos que salieron de las consultas y que acepté:

La bitácora no registraba ningún evento de negocio. Tenía 37 inicios de sesión y ninguna alta de incidencia: ocho de los nueve códigos que el sistema asentaba no existían en el catálogo, y el servicio los descarta en silencio por diseño. Apareció contando filas por tipo de evento.

El interbloqueo del botón «Resuelta»: diseño propio, los dos lados correctos por separado, imposible de ver sin usar la pantalla.

Cuatro defectos que la lectura del código no había detectado, entre ellos que asignar un técnico no movía el estado de la incidencia, y que la consulta de técnicos devolvía cero resultados con un SQL que se leía correcto.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

6 · Apéndice: las decisiones, una por una

Las secciones anteriores dicen qué decidí. Ésta dice, de cada decisión, qué había que resolver, contra qué alternativas, qué obliga a escribir en el código y cómo comprobé que quedó hecha. Es el apartado que se puede contrastar contra el repositorio: cada entrada nombra el archivo, la clase o la prueba.

Quince decisiones llevan su registro de arquitectura en docs/adr/, en formato Nygard. El criterio de significatividad está en el ADR 0001: lleva ADR lo que es costoso de revertir una vez escrito el código que depende de ello. Las que no lo son están en 6.4, porque cambian el comportamiento del sistema igual.

6.1 · La arquitectura base

C# y ASP.NET Core, en lugar del Java y Spring Boot que fija el documento · ADR 0002

Qué había que resolver. El capítulo 10 del Trabajo Final especifica Java, Spring Boot, MySQL e IIS. Había que decidir si sostener eso o apartarse.

Alternativas. Java y Spring Boot como dice el documento: significaba reescribir desde cero un andamiaje ya resuelto, en el lenguaje con el que menos recorrido tengo, sin ninguna ventaja técnica para este dominio. ASP.NET Core MVC con vistas Razor: una sola solución y menos piezas móviles, pero el módulo del cliente y el panel de administración tienen necesidades de interacción distintas. Node y TypeScript en todo el stack: es donde más práctica tengo, pero se aleja del lenguaje de la carrera y del modelo de capas que evalúa la cátedra.

Qué decidí y por qué. C# sobre .NET 9 con ASP.NET Core, React 19 con TypeScript y SQL Server 2022. El frontend separado porque el panel y el módulo del cliente son dos aplicaciones sobre la misma API, no dos secciones de la misma pantalla.

Qué obliga en el código. Seis proyectos con las dependencias cableadas para que sólo puedan ir hacia abajo: PredictIT.Domain no referencia a nadie, DAO es el único que conoce SQL, BLL orquesta y devuelve DTOs, Api sólo habla con BLL, y Service es transversal. Es la estructura del proyecto de 3.º año llevada a web.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Cómo lo verifiqué. Las referencias entre proyectos son lo que impide la violación: si Api intentara instanciar un DAO, no compila. No es una convención que haya que recordar.

Dos bases de datos con cadenas separadas · ADR 0003

Qué había que resolver. El documento ya dibuja dos diagramas entidad-relación, negocio y servicio. Faltaba decidir si esa separación conceptual se traduce en dos bases físicas o en dos grupos de tablas.

Alternativas. Una sola base con esquemas negocio y seguridad: conserva las claves foráneas y simplifica todo, pero la separación pasa a ser una convención de nombres que nada impide violar. Dos bases con sinónimos o consultas de tres partes: recupera el JOIN perforando exactamente el aislamiento que motiva la decisión.

Qué decidí y por qué. Dos bases con cadenas de conexión separadas, asumiendo la pérdida de integridad referencial entre ellas. Es el costo, y lo acepté: el punto es que un DAO de negocio no pueda alcanzar la base de seguridad ni al revés, y eso desaparece si hay una sola base.

Qué obliga en el código. Las referencias cruzadas —Equipo.id_responsable, Incidencia.id_tecnico, Mantenimiento.id_tecnico— son lógicas por GUID, sin clave foránea, documentadas como tales en el esquema. El motor ya no las puede garantizar, así que su validación se hace en la BLL antes de persistir. Hay dos ayudantes, SqlHelper y SqlHelperSeguridad, y cada uno recibe su cadena y no conoce la otra.

Cómo lo verifiqué. Pruebas de integración contra las dos bases del contenedor, con una transacción que revierte.

ADO.NET con DAO explícito, no Entity Framework · ADR 0004

Qué había que resolver. Cómo se accede a los datos. Es la decisión que más código repetitivo produce o evita.

Alternativas. EF Core con DbContext: vacía de contenido la capa DAO y el Unit of Work, que es justamente lo que se evalúa. Dapper: buen punto medio —SQL a la vista, mapeo automático—, pero ese mapeo automático es precisamente el IMapper<T> que hay que mostrar escrito. EF Core sólo para los catálogos: dos modelos mentales de acceso a datos en el mismo sistema, sin ganancia.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Qué decidí y por qué. ADO.NET con SQL parametrizado y mapeo explícito, asumiendo el código repetitivo. Un ORM esconde exactamente lo que la materia pide mostrar.

Qué obliga en el código. `IGenericDao<T>` como contrato base; `IObjectMapper<T>` por entidad, que traduce `SqlDataReader` a objeto de dominio; `FactoryDao` y `FactoryBusiness` como fábricas abstractas; `SqlTransactRepository` como unidad de trabajo cuando una operación toca varias tablas. Todo parámetro entra como `SqlParameter`: no hay una sola concatenación de SQL en las cinco capas, que es lo que cierra la inyección por construcción y no por revisión.

Multi-organización desde el modelo de datos · ADR 0005

Qué había que resolver. El diagrama entidad-relación del documento no tiene `Organizacion`, pero el plan de negocio vende un Plan Partner multi-cliente. Sin la entidad, ese plan no se puede sostener.

Qué decidí y por qué. `Organizacion` como raíz del modelo, y `id_organizacion` en toda tabla de negocio.

Qué obliga en el código, y es la parte que importa. El filtro por organización se aplica en la capa DAO, tomando la organización del contexto de sesión y no del pedido HTTP. Es deliberado: puesto en el controlador, un endpoint nuevo puede olvidarse de filtrar y nadie se entera hasta que un cliente ve los datos de otro. Puesto en el DAO, no hay forma de consultar sin filtrar. Un controlador no puede pedir datos de otra organización porque no tiene cómo expresarlo.

Identidad propia, sin gestor externo · ADR 0014

Qué había que resolver. Si incorporar un gestor de identidad federada —del tipo Keycloak— o resolver la identidad y la autorización dentro del sistema.

Qué decidí y por qué, y el motivo principal no es técnico. No se incorpora. El modelo de familias y patentes, con el composite anidado y su guarda de ciclos, es contenido de la materia. Delegarlo en un gestor externo lo convertiría en un archivo de configuración: los permisos pasarían a ser roles y ámbitos de un producto de terceros, y lo que la cátedra evalúa dejaría de estar escrito en el sistema.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Qué obliga en el código. Resolver si un usuario tiene un permiso no es leer una fila: Familia_Familia permite familias dentro de familias, así que el chequeo es un Composite con recorrido en profundidad y guarda de ciclos. El filtro [RequierePatente("...")] de la API se apoya en ese recorrido.

6.2 · El dominio y el motor predictivo

Proveedor de IA conmutable, con asignación de respaldo · ADR 0006

Qué había que resolver. El sistema asigna el técnico con inteligencia artificial, y no hay crédito de API. Había que decidir cómo construirlo sin que esa falta bloqueara el desarrollo ni volviera el caso de prueba irreproducible.

Alternativas. Llamar a la API directamente desde la BLL: ata el sistema a un proveedor, hace las pruebas dependientes de la red y vuelve el CP-14 no reproducible. Simular sólo en las pruebas con un objeto simulado: el simulacro prueba el código de prueba, no el sistema. Entrenar un modelo propio: fuera del alcance y sin volumen que lo justifique —una PyME con 50 equipos no genera un conjunto de entrenamiento—. Una cola asíncrona: elimina la latencia del camino crítico, pero agrega infraestructura desproporcionada y rompe la expectativa del CU-007, que asigna al registrar.

Qué decidí y por qué. Adapter: IProveedorIA con dos implementaciones, ProveedorIAClaude contra la API de Anthropic y ProveedorIASimulado, heurística determinística sin red y con capacidad de simular fallas y demoras a pedido. El modo simulado no es un andamio de pruebas: es un modo de operación legítimo, y es como se demuestra el sistema sin crédito.

Qué obliga en el código. El camino de respaldo no es un catch vacío. Ante error o tiempo agotado: se asigna al técnico de menor carga, se deja tipo_asignacion = 'RESPALDO', se marca la incidencia para revisión y se asienta el evento en la bitácora.

Cómo lo verifiqué. El CP-14 fuerza la caída del proveedor y comprueba las cuatro consecuencias. Es reproducible porque la falla se provoca, no se espera.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Trabajo Final de Ingeniería		Docentes:		Fecha: 13/06/2026
			Romero, Martin - Weingand, Gastón		
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1		
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026	
	Sistema de Mantenimiento predictivo de equipos informáticos				

Reglas predictivas parametrizadas, resueltas con estrategias · ADR 0007

Qué había que resolver. Cinco tipos de regla que miran cosas distintas —recurrencia de fallas, mantenimiento vencido, acumulación, antigüedad, garantía próxima a vencer— y que tienen que poder ajustarse sin recompilar.

Qué decidí y por qué. Strategy. ReglaAlerta guarda el tipo que discrimina la estrategia y condicion con los parámetros en JSON, de modo que agregar una regla es agregar una clase y una fila, no tocar el evaluador.

Tres decisiones de comportamiento que no son obvias y cambian el resultado:

La fecha de referencia entra por parámetro; el motor no lee el reloj. Sin esto no existe caso de prueba posible: el resultado cambiaría según el día en que se corran las pruebas. Es la decisión que hace verificable todo el capítulo.

Una regla que no se puede evaluar por falta de datos se excluye del promedio ponderado, en lugar de aportar cero. Aportando cero, el equipo con la ficha incompleta aparecería *menos* riesgoso que el que está bien cargado, que es exactamente al revés de lo que corresponde.

La criticidad del equipo multiplica el puntaje, no lo suma. Sumando, un servidor y una impresora con el mismo historial terminan a distancia fija; multiplicando, la criticidad pesa sobre todo el resto.

El motor se dispara solo · ADR 0011

Qué había que resolver. El motor evaluaba al registrar una incidencia. Una alerta que depende de que alguien cargue algo no es predictiva: un equipo cuya garantía vence mañana no genera ningún hecho que dispare la evaluación.

Qué decidí y por qué. Dos disparadores, porque las condiciones aparecen de dos maneras distintas: por hecho, al registrar una incidencia o un mantenimiento, que son los momentos en que cambian los datos que las reglas miran; y por tiempo, un barrido programado, para las condiciones que se cumplen por el paso del calendario y no por un hecho.

Qué obliga en el código. Un servicio alojado en la API, con el intervalo configurable, y la fecha de referencia siempre por parámetro para que el barrido sea igual de verificable que la evaluación por hecho.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

La máquina de estados de la incidencia vive en la BLL · ADR 0012

Qué había que resolver. El catálogo EstadoIncidencia tiene siete estados y sólo dos eran alcanzables. Las columnas diagnostico, solucion y fecha_resolucion existían, el detalle las devolvía, y ninguna consulta del sistema las escribía. Sin fecha de resolución no hay tiempo de reparación, no hay cumplimiento del acuerdo de servicio de cuatro horas que el documento promete, y el tablero no puede calcular el tiempo medio.

Alternativas. En la base con una restricción CHECK: las transiciones quedarían fuera del alcance de las pruebas. En el frontend deshabilitando botones: no es una regla, es una sugerencia, y cualquiera que llame a la API la saltea.

Qué decidí y por qué. En la capa de negocio, en un solo lugar, con la tabla de transiciones como diccionario en código. Es el único sitio desde donde se puede recorrer entera en una prueba.

Cómo apareció el problema, que es lo que quiero mostrar. No lo encontré leyendo el código: busqué patentes declaradas que ningún endpoint exige. La patente INCIDENCIA_ATENDER estaba en la base y nadie la usaba. Es la misma técnica con la que encontré la bitácora sin eventos de negocio y el motor que no se disparaba solo: comparar lo que el sistema declara contra lo que el sistema hace.

Reportes en PDF generados en el servidor · ADR 0013

Qué había que resolver. El requisito pide reportes exportables. La propuesta inicial era CSV, por ser lo más barato y sin dependencias.

Qué decidí y por qué. PDF generado en el servidor con QuestPDF. Un CSV no lleva encabezado, ni la fecha de emisión, ni quién lo generó, ni los filtros aplicados impresos, y no se ve como un entregable. Y en el servidor y no en el navegador porque un reporte tiene que poder explicarse solo: si lo arma el navegador, lo firma el navegador.

6.3 · Seguridad y operación

Cifrado reversible, separado del hash de contraseñas · ADR 0010

Qué había que resolver. El sistema guarda dos cosas parecidas y distintas: la contraseña de un usuario y la clave de API de la organización.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

Qué decidí y por qué. Dos mecanismos que conviven porque resuelven problemas opuestos. Una contraseña no hace falta recuperarla: el sistema sólo necesita verificarla, y poder recuperarla sería una debilidad. Una credencial de un tercero sí hay que recuperarla, porque hay que enviarla en cada llamada. Usar hash para la clave de API sería inútil; usar cifrado para las contraseñas sería peor que inútil.

Qué obliga en el código. PBKDF2-HMAC-SHA256 con 600.000 iteraciones —lo que recomienda OWASP— y el formato `PBKDF2$iteraciones$salt$hash`, para la contraseña. AES-256-GCM con la clave fuera del código, para la de API. Ningún endpoint devuelve la clave de API, ni truncada: lo único que la pantalla puede mostrar es si hay una y de dónde viene.

Patrones de resiliencia, acotados a dos puntos · ADR 0009

Qué decidí y por qué. No se adopta una política general de resiliencia: cada patrón responde a un hallazgo concreto. Un disyuntor delante de `IProveedorIA`, con tres estados, porque es la única llamada a un tercero por red; y un mamparo para que la lentitud de esa llamada no consuma el grupo de conexiones que necesita el resto del sistema.

El nivel de verificación es superior al estándar, a propósito · ADR 0008

Qué había que resolver. La asistencia por IA produce código más rápido de lo que se puede revisar leyendo. Esa velocidad hay que compensarla con verificación o se convierte en deuda.

Qué decidí. Tres controles obligatorios, ejecutables por comando y verificados antes de cada entrega, ninguno «para si hay tiempo»: analizadores de .NET en modo estricto con `AnalysisMode=All` y `TreatWarningsAsErrors=true` —una advertencia rompe la compilación—, revisión de dependencias vulnerables, y pruebas de carga con perfil definido.

Métricas en formato Prometheus, sin stack propio · ADR 0015

Qué decidí y por qué. El sistema expone sus métricas; no incorpora Prometheus ni Grafana. La aplicación expone y la infraestructura recolecta: es el reparto habitual, y montar un stack de recolección para un sistema que corre en una notebook sería instrumentar lo que no hay. El endpoint no existe si no hay token configurado, porque dice qué rutas hay y cuánto tarda cada una.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	Fecha: 13/06/2026
	Alumno: Pettini, Patricio Ezequiel		Legajo: B00072710-T1	
	Sede: Norte	Comisión: 5 - A	Turno: Noche	
	Sistema de Mantenimiento predictivo de equipos informáticos			Versión: 2.0

6.4 • Decisiones de código que no llevan ADR

No son de arquitectura —no son costosas de revertir— pero cambian el comportamiento y cada una tiene su motivo.

Decisión	Por qué
plan_comercial y Respaldo en lugar de plan y Backup	Las dos son palabras reservadas de T-SQL. Es el tipo de detalle que no se descubre diseñando: se descubre cuando la consulta falla
CROSS APPLY en lugar de SUM(CASE WHEN EXISTS (subconsulta))	SQL Server no admite una subconsulta correlacionada dentro de una función de agregación. Devolvía 500
Índice único filtrado para el número de factura	UNIQUE (organizacion, numero) con numero nulo mientras es borrador: SQL Server trata el NULL como un valor más, así que la organización admitía un solo borrador. Apareció facturando dos meses seguidos, usando el sistema
El asiento en bitácora reintenta ante fallos transitorios	La excepción se traga a propósito, para no abortar la operación de negocio. Sin reintento eso dejaba agujeros de auditoría justo en los momentos de más actividad
.cuerpo > * { flex: none } en el frontend	Un contenedor flex encoge a sus hijos, y una tarjeta con overflow: hidden tiene mínimo automático cero: se aplastaba a la altura de sus bordes. Medía 2 px con 400 px de contenido adentro
El tamaño de página lo decide la vista y no los filtros	Limpiar los filtros en la vista de tablero la achicaba de 200 a 20 tarjetas sin que nadie lo pidiera

6.5 • Cómo audito la construcción

Es la parte que no se ve en una lista de decisiones, y es la que hace que las decisiones se sostengan en el tiempo.

La técnica. Comparar lo que el proyecto declara contra lo que el proyecto mide. No leer el código buscando errores: preguntarle al sistema por sus propios datos. Con eso aparecieron la bitácora que tenía 37 inicios de sesión y ninguna alta de incidencia —ocho de los nueve códigos que el sistema asentaba no existían en el catálogo y el servicio los descartaba en silencio—, la patente declarada que ningún endpoint exigía, el motor predictivo que no se disparaba solo, el recuento

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final de Ingeniería		Docentes: Romero, Martín - Weingand, Gastón	
	Alumno: Pettini, Patricio Ezequiel			Legajo: B00072710-T1
	Sede: Norte	Comisión: 5 - A	Turno: Noche	Año: 2026
	Sistema de Mantenimiento predictivo de equipos informáticos			
				Fecha: 13/06/2026
				Versión: 2.0

de endpoints que decía 57 cuando había 78, y veinticuatro capturas de pantalla anteriores al rediseño del frontend metidas en el documento.

Los controles automatizados. Veintidós, en `docs/_verificar-alineacion.py`. Cada uno compara un número que vive en varios archivos: los precios de los planes están en el documento, en el seed de la base, en la planilla y en el plan de negocio, y cuando uno cambia y los otros no, nadie se entera hasta que alguien compara dos archivos en la defensa.

Las pruebas. 361 de backend y 91 de frontend. Los casos de prueba CP-01 a CP-15 están automatizados, incluido el CP-14, que fuerza la caída del servicio de IA.

Y una regla que me impuse: ningún control se da por bueno sin verlo fallar. Un control que no puede ponerse en rojo da confianza falsa, que es peor que no tenerlo. Lo comprobé rompiendo a propósito lo que cada uno vigila, antes de confiar en él. Dos ejemplos de lo que eso encontró:

El control que verifica el resumen del propio apéndice nunca podía fallar: el patrón cortaba en la primera palabra, así que de «cuarenta y siete» leía «cuarenta», no lo encontraba en su tabla y se salteaba la comparación en silencio. Lo descubrí cambiando el número a mano y viendo que seguía diciendo «ok».

El control que busca vínculos cruzando cajas en los diagramas buscaba `LineStyle == 7`, y Enterprise Architect normaliza ese valor a 3 al guardar: los tres diagramas entidad-relación y los tres de clases se medían como si fueran de línea recta, y daban treinta y cinco cruces que en el dibujo no existen.